DATAPULSE

System Architecture

A Hybrid Observability and Business-Event Analytics Platform

Architecture article covering ingestion, asynchronous processing, storage, search, observability, security, deployment, and performance validation.

Architecture Design Document

Version 1.0 | 9 October 2026

Design principle: build for measurable reliability and scale, while keeping the initial implementation reproducible and intentionally simple.

Introduction

DataPulse is a hybrid observability and business-event analytics platform designed to ingest, process, search, and analyze large volumes of application telemetry and business events. The platform combines operational observability with business analytics, allowing engineering teams to investigate application failures, monitor service performance, track background processing, and understand business activity through a unified interface.

The architecture is designed around a data-intensive workload, with an initial benchmark target of one million events and a roadmap toward processing five to ten million events. Rather than relying on a single database or processing every request synchronously, DataPulse separates event ingestion, background processing, transactional state management, analytical storage, search, and visualization into distinct responsibilities.

The initial implementation will use synthetic data to establish a reproducible testing environment. Public datasets and telemetry generated by DataPulse's own services will be introduced as the platform matures. The primary architectural objective is to build a reliable and observable system that can be measured, tested under load, and extended without introducing unnecessary complexity.

Architectural Overview

DataPulse follows a layered architecture consisting of data sources, an API and ingestion layer, an asynchronous processing layer, specialized storage systems, a presentation layer, and an observability infrastructure. Each layer has a defined responsibility and communicates with other components through explicit interfaces.

The data sources generate application events, API telemetry, worker activity, error records, and business events such as user actions or order-related activities. These events are submitted to the DataPulse ingestion API, implemented using Django and Django REST Framework. The API handles authentication, authorization, schema validation, request-size limits, rate limiting, and ingestion-job creation.

After an event batch has passed validation, DataPulse records the relevant ingestion metadata and processing state in PostgreSQL. The accepted work is then dispatched for asynchronous processing through Celery, using Redis as the message broker. Background workers process events independently of the original HTTP request, allowing the ingestion API to respond without waiting for all downstream processing and indexing operations to finish.

Processed events are routed to ClickHouse for analytical queries and Elasticsearch for full-text search and event exploration. PostgreSQL remains responsible for transactional application data, including users, organizations, projects, permissions, ingestion jobs, and audit records. The frontend, built with Next.js, React, and TypeScript, provides dashboards, event search, processing-job visibility, and project-management functionality through the DataPulse API.

The entire system is instrumented to expose its own performance, failures, and resource utilization. Prometheus-compatible metrics, centralized logs, distributed traces, Grafana dashboards, and Sentry error reporting provide visibility into the application's behavior during normal operation and under load.

Event Ingestion and API Management

The ingestion layer is the entry point for external event producers and internal services. Django REST Framework provides the API endpoints through which the synthetic generator, public-dataset importers, and instrumented applications submit individual events or batches of events.

When a request arrives, the API authenticates the caller, verifies project-level permissions, validates the event structure, and applies configured request and rate limits. Each event follows a canonical schema containing fields such as event ID, schema version, event timestamp, ingestion timestamp, project ID, event type, source, service, severity, trace ID, and a flexible attributes object.

Separating the event timestamp from the ingestion timestamp makes it possible to distinguish when an event occurred from when DataPulse received it. This distinction is important when measuring ingestion delay, investigating delayed telemetry, and analyzing events arriving out of chronological order.

The API records an ingestion job in PostgreSQL so that clients can track the progress and outcome of submitted batches. Accepted requests are passed to the asynchronous processing layer rather than being held open while analytical aggregation and search indexing take place. This approach protects API responsiveness as event volume increases.

The ingestion contract must clearly distinguish between events that are rejected during validation and events that have been accepted but are still processing. An accepted event is not necessarily immediately visible in every downstream store. DataPulse therefore exposes processing status and defines measurable targets for how quickly accepted events become available for analytics and search.

Asynchronous Processing with Celery and Redis

Event processing is handled by Celery workers, with Redis providing the task broker and selected coordination capabilities. This separation allows DataPulse to absorb temporary increases in incoming traffic without requiring every event to be processed within the lifetime of an HTTP request.

When the API accepts an ingestion job, it dispatches a task containing a reference to the job or its stored batch rather than unnecessarily duplicating large payloads in queue messages. Workers retrieve the required data, normalize event fields, apply processing rules, detect duplicates, and prepare records for downstream storage.

Celery workers operate with bounded concurrency and controlled batch sizes. These limits prevent a sudden increase in ingestion traffic from consuming excessive memory or overwhelming PostgreSQL, ClickHouse, and Elasticsearch. Queue depth, task duration, processing throughput, retry counts, and worker health are monitored to determine when the system is falling behind.

Redis serves multiple purposes, but these responsibilities remain logically separate. Its primary role in the initial architecture is to broker Celery tasks. It may also support short-lived caching, rate limiting, and coordination where appropriate. It is not treated as the authoritative store for ingestion jobs or permanent event records.

Transient failures are handled through controlled retries with exponential backoff. For example, a temporary connection failure while writing to a downstream store should not immediately cause an accepted batch to be discarded. At the same time, retries must be bounded, and repeatedly failing tasks must become visible for investigation or controlled reprocessing.

The processing layer is designed around at-least-once task delivery and idempotent processing effects. This means a task may run more than once, but repeated execution should not unintentionally create duplicate business effects. DataPulse does not assume that Celery alone guarantees exactly-once processing across multiple independent databases.

Data Storage and Database Responsibilities

DataPulse uses PostgreSQL, ClickHouse, and Elasticsearch because the platform supports different access patterns that are not equally well served by a single storage engine.

PostgreSQL is the transactional system of record for application metadata and workflow state. It stores user accounts, organizations, projects, access permissions, ingestion-job records, processing status, and relevant audit information. Django's ORM provides the primary interface for these entities, while database constraints and transactions protect consistency.

As event volume grows, database performance will depend on appropriate schema design, indexing, query optimization, connection management, and transaction boundaries. PostgreSQL indexes will be selected according to actual query patterns rather than added indiscriminately. Large analytical scans of event history will be directed toward ClickHouse instead of being performed repeatedly against transactional tables.

ClickHouse is the analytical data store. It is designed to support efficient aggregations over large event datasets, including event counts by time interval, error rates by service, latency distributions, event volumes by type, and other dashboard queries. Its column-oriented storage and analytical query engine make it a suitable choice for workloads involving millions of records and repeated aggregate calculations.

The ClickHouse data model will prioritize time-based analysis, efficient filtering, and aggregation. Appropriate partitioning, ordering keys, retention policies, and batch-insertion strategies will be selected based on expected access patterns and benchmark results. The design will also account for duplicate delivery and the distinction between event time and ingestion time.

Elasticsearch provides full-text search and flexible filtering for individual events. Engineers should be able to search error messages, filter by service or severity, narrow results to a time range, and investigate related event attributes. Search indexes will be optimized for these investigative workloads rather than used as the primary source of transactional application state.

Because these storage systems have independent write paths, DataPulse must account for temporary differences between them. An event may become available in ClickHouse before it appears in Elasticsearch, or one destination may temporarily fail while another succeeds. The architecture therefore tracks processing and delivery outcomes sufficiently to support retries, monitoring, and reconciliation. The API and dashboard must communicate realistic consistency expectations instead of assuming all stores update simultaneously.

Event Identity, Idempotency, and Consistency

Reliable event processing requires an explicit event-identity strategy. Each event carries a stable event ID that remains unchanged across retries and reprocessing attempts. This identity allows workers and downstream write paths to recognize repeated deliveries and apply deduplication rules appropriate to each storage system.

Task-level idempotency and event-level deduplication address related but different problems. Task-level idempotency prevents rerunning an ingestion job from unintentionally repeating its effects, while event-level deduplication helps prevent the same logical event from being counted or indexed multiple times.

The architecture will not rely solely on application-level checks such as querying for an existing event before every write, because concurrent workers can race and both observe that the event is absent. Where necessary, database constraints, durable processing records, deterministic batch identifiers, or storage-specific deduplication mechanisms will provide stronger guarantees. ClickHouse and Elasticsearch require particular care because their deduplication behavior differs from PostgreSQL's transactional uniqueness constraints.

A downstream write failure is handled independently where practical. If ClickHouse accepts a batch but Elasticsearch rejects it, DataPulse should retry the search-indexing operation without blindly repeating all successful work. Processing status and destination-specific outcomes make this possible and provide a basis for reconciliation.

The platform adopts eventual consistency across analytical and search stores. Temporary delays are expected, but they must be measurable and recoverable. The initial performance objective is for at least 95 percent of accepted events to become available for analytics within ten seconds under the specified sustained benchmark load. Search visibility will be measured separately so that indexing delays are not hidden by aggregate processing metrics.

Frontend and Query Access

The presentation layer is built with Next.js, React, and TypeScript. It provides a unified interface for viewing operational metrics, analyzing business events, searching event records, monitoring ingestion jobs, and managing projects.

The frontend communicates with the Django REST Framework API rather than connecting directly to databases. This preserves authentication and authorization boundaries, centralizes query validation, and allows the backend to coordinate requests to PostgreSQL, ClickHouse, and Elasticsearch.

Analytical dashboards retrieve aggregated results from ClickHouse, while the event explorer uses Elasticsearch for text search and filtering. Project settings and job-status views rely primarily on PostgreSQL-backed API endpoints. The API enforces project-level access checks so that users can access only the data permitted by their organization and project membership.

The interface will distinguish between ingestion acceptance, processing completion, analytical visibility, and search visibility. This distinction prevents a submitted batch from appearing fully complete when some downstream work is still pending.

Dashboard queries will be designed around explicit time ranges, bounded result sizes, and appropriate filtering. Pagination and query limits will protect the backend from expensive requests, while caching may be introduced for frequently requested results once measurements show that it is useful.

Observability and Operational Monitoring

DataPulse must be observable as a system in its own right. Since it processes telemetry and business events, failures in its ingestion and processing pipeline can affect the reliability of the very data the platform is intended to analyze. Operational monitoring must therefore include independent visibility into the infrastructure and application services.

Prometheus collects application and infrastructure metrics, including request latency, request rates, HTTP error rates, worker utilization, queue depth, task failures, database health, ingestion throughput, and event-visibility delays. Mimir can serve as a scalable long-term metrics backend when the project requires a dedicated metrics-storage layer. For the initial local implementation, the decision to run both Prometheus and Mimir will depend on whether the added operational complexity provides enough learning or functional value.

Loki centralizes application and worker logs, allowing engineers to investigate failed jobs and correlate error messages with relevant services and time ranges. Tempo stores distributed traces that reveal how requests and background tasks move through the system. OpenTelemetry instrumentation and compatible exporters will connect application-level traces, metrics, and logs where supported.

Grafana provides dashboards and alerting for the metrics, logs, and traces collected by these systems. Sentry complements this stack by reporting application exceptions and relevant performance issues, helping developers identify errors with useful diagnostic context.

Observability will cover both technical health and data-pipeline health. Technical indicators include API latency, memory consumption, database connections, worker health, and error rates. Pipeline indicators include accepted events per second, queue backlog, processing duration, retry rates, dead-letter or permanently failed jobs, destination-specific write failures, and time to analytical or search visibility.

These signals will be used to evaluate the system against the report's performance targets. A benchmark is not considered successful merely because the API accepts events quickly; the full processing path must also maintain acceptable latency, bounded resource consumption, and recoverability under failure.

Security and Tenant Isolation

DataPulse will support organization- and project-level access boundaries. Authentication establishes the identity of a caller, while authorization determines which projects and event data that caller can access. Tenant restrictions must be enforced in API query paths and background processing, not only in frontend navigation.

Ingestion endpoints will apply request-size limits, schema validation, rate limiting, and explicit handling of malformed input. Credentials and service configuration will be supplied through environment-specific secrets rather than committed to source control. Logs and traces will avoid unnecessarily recording secrets or sensitive event payloads.

The synthetic-data-first strategy reduces the risk of exposing real customer information during early development. When public datasets or application telemetry are introduced, each source will be reviewed for licensing, privacy implications, data quality, and suitability for the canonical event schema.

Retention policies will define how long event records and operational data remain available. These policies will account for the different storage systems and their deletion semantics so that event retention and tenant-isolation requirements are not undermined by secondary indexes or stale derived data.

Deployment and Development Workflow

The initial environment will use Docker Compose to run the application and its supporting infrastructure consistently across development machines. The environment will include the Django API, Celery workers, Redis, PostgreSQL, ClickHouse, Elasticsearch, the Next.js frontend, and the observability components required for the current development stage.

Services will have appropriate health checks, persistent storage where required, and explicit configuration for service dependencies. Application readiness will be distinguished from container startup: a process being alive does not necessarily mean that its database or message broker is ready to accept work.

The development environment will be designed to be reproducible, allowing engineers to start the platform, generate synthetic events, inspect processing jobs, run queries, and reproduce failures without relying on production infrastructure. Since the full observability stack can consume substantial local resources, optional services will be introduced in stages instead of requiring every component to run from the first day.

GitHub Actions will provide continuous integration. Changes will be checked through formatting, linting, static analysis, automated tests, and relevant integration checks. Container images can then be built and verified before being promoted through staging and production deployment workflows. Secrets, environment-specific configuration, database migrations, and rollback procedures will be handled explicitly rather than embedded in build scripts.

The first delivery target is a reliable local development environment and automated test pipeline. More advanced deployment infrastructure will be introduced only when the application and its operational

requirements justify it.

Performance, Scalability, and Validation

The architecture is designed around measurable requirements rather than assumptions about scalability. The first major benchmark will use one million events, with further testing toward five to ten million events as resource capacity permits. The ingestion path will be tested at a sustained rate of 500 events per second for fifteen minutes and a burst rate of 1,000 events per second for sixty seconds.

At the defined target load, the initial objectives include a p95 ingestion API latency below 300 milliseconds for batches of up to 100 small events, p95 dashboard-query latency below two seconds for representative analytical queries, and p95 search latency below one second for representative event searches. At least 95 percent of accepted events should become available for analytics within ten seconds under sustained target load.

These figures are engineering targets to validate, not claims of achieved production performance. Benchmark results will document the test environment, event size, batch configuration, concurrency, database settings, resource usage, and failure conditions. This makes performance comparisons meaningful and helps distinguish application bottlenecks from hardware or configuration constraints.

Testing will include more than successful ingestion. The platform will be exercised under duplicate delivery, malformed events, worker restarts, temporary database unavailability, failed indexing, retry exhaustion, and queue backlog. Memory usage and queue growth will be observed during sustained processing to verify that the system applies backpressure rather than allowing resource consumption to grow without control.

The results will guide subsequent optimization. Likely areas include PostgreSQL query plans and indexes, ClickHouse insert batching and table ordering, Elasticsearch indexing throughput, Celery concurrency, connection pooling, and frontend query patterns. Each optimization will be justified by measurements and regression tests.

Conclusion

DataPulse uses a layered, asynchronous architecture to combine application observability with business-event analytics. Django and Django REST Framework manage ingestion and application access, Celery and Redis support background processing, PostgreSQL maintains transactional state, ClickHouse powers analytical queries, and Elasticsearch enables event search. Next.js provides the user interface, while Prometheus, Grafana, Loki, Tempo, Mimir, and Sentry provide operational visibility.

The architecture acknowledges that high-volume data processing involves more than choosing fast databases. It requires explicit event identity, controlled concurrency, reliable task execution, independent recovery of downstream writes, tenant isolation, measurable consistency, and disciplined operational monitoring.

By starting with synthetic data, establishing measurable acceptance criteria, and validating the complete event lifecycle before optimizing individual components, DataPulse can evolve from a reproducible local project into a credible demonstration of production-oriented engineering. The next design stage will formalize the end-to-end event flow, job-state transitions, data contracts, and synchronous versus asynchronous boundaries before implementation begins.